

- Given a or b, if a fails on the input, do we *always* want to run b, or are there cases where we might not want to? If there are such cases, can you think of additional combinators that would allow the programmer to specify when or should consider the second parser?
- How do you want to handle reporting the *location* of errors?
- Given a or b, if a and b both fail on the input, might we want to support reporting both errors? And do we *always* want to report both errors, or do we want to give the programmer a way to specify which of the two errors is reported?

We suggest you continue reading once you're satisfied with your design. The next section works through a possible design in detail.

Combinators specify information

In a typical library design scenario, where we have at least some idea of a concrete representation, we often think of functions in terms of how they will affect this representation. By starting with the algebra first, we're forced to think differently—we must think of functions in terms of *what information they specify* to a possible implementation. The signatures determine what information is given to the implementation, and the implementation is free to use this information however it wants as long as it respects any specified laws.

9.5.1 A possible design

Now that you've spent some time coming up with some good error-reporting combinators, we'll work through one possible design. Again, you may have arrived at a different design and that's totally fine. This is just another opportunity to see a worked design process.

We'll progressively introduce our error-reporting combinators. To start, let's introduce an obvious one. None of the primitives so far let us assign an error message to a parser. We can introduce a primitive combinator for this, `label`:

```
def label[A](msg: String)(p: Parser[A]): Parser[A]
```

The intended meaning of `label` is that if `p` fails, its `ParseError` will somehow incorporate `msg`. What does this mean exactly? Well, we could just assume type `ParseError` = `String` and that the returned `ParseError` will *equal* the label. But we'd like our parse error to also tell us *where* the problem occurred. Let's tentatively add this to our algebra:

```
case class Location(input: String, offset: Int = 0) {
  lazy val line = input.slice(0,offset+1).count(_ == '\n') + 1
  lazy val col = input.slice(0,offset+1).lastIndexOf('\n') match {
    case -1 => offset + 1
    case lineStart => offset - lineStart
  }
}

def errorLocation(e: ParseError): Location
def errorMessage(e: ParseError): String
```